

# Stock Exchange / High-Frequency Trading System - End-to-End System Design

---

Goal: practice one complete E2E trading system from problem understanding to HLD, LLD, machine coding, traffic spike handling, and global scale, with matching engine, order book, market data, and low-latency execution at its core.

---

## How To Use This File

---

- Treat this as the repeatable pattern for exchange, matching engine, and low-latency trading systems.
  - Start broad with requirements and scale, then zoom into order gateway, matching engine, order book, market data fanout, risk checks, clearing/settlement, LLD, and coding.
  - In interviews, do not recite everything. Pick the parts that match the interviewer's signal.
  - For trading systems, optimize determinism, latency, order correctness, fairness, risk safety, and auditability before raw feature breadth.
- 

## Starter Learning Path

---

Read this file in four passes:

1. First pass: understand the product goal, core requirements, and the user-facing workflow.
2. Second pass: trace the main read/write path through the high-level design.
3. Third pass: study the data model, scaling choices, failures, and trade-offs.
4. Fourth pass: practice the LLD, machine-coding layer, and final interview playbook without looking.

What a starter should master first:

- The one-line purpose of the system.
- The core entities and APIs.
- The main request flow.
- The storage choice and why it fits.
- The biggest bottleneck.
- The failure that most affects users.
- The trade-off you would defend in an interview.

Gold-level self-check:

- You can draw the architecture from memory in 5 minutes.
  - You can explain the happy path and one failure path clearly.
  - You can justify consistency, latency, availability, and cost choices.
  - You can name what you would simplify for an MVP and what you would add at scale.
  - You can answer follow-ups about spikes, retries, idempotency, observability, and data growth.
-

# Master Checklist For This Problem

| Layer                 | Interview signal                             | Trading system focus                                                   |
|-----------------------|----------------------------------------------|------------------------------------------------------------------------|
| Problem understanding | Can clarify trading lifecycle                | submit order, risk check, match, fill, market data, settle             |
| HLD                   | Can design low-latency deterministic systems | gateway, risk, matching engine, order book, market data, clearing      |
| LLD                   | Can model deterministic components           | Order , OrderBook , PriceLevel , Trade , MatchingEngine , Position     |
| Machine coding        | Can implement critical path                  | limit order book, price-time matching, cancel/replace, fill generation |
| Traffic spikes        | Can protect the exchange                     | open/close auction, news shock, quote storms, circuit breakers         |
| Global scale          | Can reason across venues                     | per-symbol sharding, sequencer, cross-venue market data, colocation    |

## 1. Problem Understanding

### 1.1 Functional Requirements

Core requirements:

- Traders can submit orders (market, limit, and cancel/replace).
- System runs pre-trade risk checks before an order reaches the book.
- A matching engine matches buy and sell orders by price-time priority.
- System generates trades (fills) and updates the order book.
- System publishes market data (order book updates, last trade, top of book).
- Traders can cancel or modify resting orders.
- System records every order, trade, and state change for audit.
- Trades flow to clearing and settlement.

Optional requirements to clarify:

- Are we building a full exchange, a broker order-management system, or an internal HFT engine?
- Which order types are in scope (market, limit, IOC, FOK, stop, iceberg)?
- Is this a single symbol or a full multi-symbol venue?
- Is auction (open/close) matching in scope, or only continuous trading?
- Do we support level-2 (full depth) market data or only top of book?
- Are margin, short selling, and derivatives in scope?

Out of scope unless interviewer asks:

- Full clearing house and CCP netting internals.
- Full regulatory reporting engine (e.g., trade surveillance, MiFID/Reg NMS detail).
- Full margin and collateral risk models.
- Full smart order routing across external venues.

## 1.2 Non-Functional Requirements

---

Determinism and correctness:

- Matching must be deterministic and reproducible from the order sequence.
- Price-time priority (fairness) must never be violated.
- No order is lost, duplicated, or matched twice.

Latency:

- Order-to-ack and order-to-match are single-digit microseconds to milliseconds depending on tier.
- Market data fanout must be low-latency and fair across subscribers.
- Tail latency (p99/p99.9) matters more than average latency.

Reliability and auditability:

- Every event is sequenced, durable, and replayable.
- The engine must recover to an exact state after failure.
- Full audit trail for regulators.

## 1.3 Constraints

---

- The matching engine is effectively single-writer per symbol for determinism.
- Fairness requires strict ordering; concurrency must not reorder same-priority intent.
- Bursts are extreme and correlated (news, open, close).
- Market data must be fair: no subscriber should get a structural time advantage inside the system.
- Risk checks must be fast but cannot be skipped.
- Recovery must reproduce the exact same book state.

## 1.4 Scale Assumptions

---

Example interview-scale assumptions:

| Metric                          | Assumption                     |
|---------------------------------|--------------------------------|
| Symbols                         | thousands to tens of thousands |
| Peak order messages/sec         | millions/sec across venue      |
| Peak per-hot-symbol msgs/sec    | hundreds of thousands/sec      |
| Trades/day                      | tens to hundreds of millions   |
| Market data updates/sec         | millions/sec                   |
| Order-to-ack latency (hot tier) | microseconds                   |
| Order-to-match latency          | microseconds to low ms         |
| Engine availability target      | 99.99%+ during market hours    |

## 1.5 Capacity Math

---

Back-of-the-envelope:

- Message rate is dominated by adds/cancels, not fills; cancel rate is often much higher than trade rate.
- A single hot symbol can concentrate a large share of total traffic.
- Order book memory is small per symbol but must be cache-friendly for latency.

- Market data fanout is a write-amplification problem: one book change fans out to many subscribers.
- The sequencer/log throughput bounds the whole system.

Useful interview numbers:

| Item                            | Rough value                     |
|---------------------------------|---------------------------------|
| Cancel/replace to trade ratio   | often 10:1 or higher            |
| Order book depth kept in memory | full for active symbols         |
| Sequenced event log             | append-only, replayable         |
| Risk check budget               | sub-microsecond to microseconds |
| Snapshot interval for recovery  | seconds                         |
| Market data feed types          | incremental + periodic snapshot |

## 1.6 Clarifying Questions To Ask

- Are we the exchange (matching venue) or a participant (HFT/broker)?
- Continuous trading only, or auctions too?
- Which order types and time-in-force?
- Top-of-book or full-depth market data?
- What are the latency and fairness guarantees?
- What are the recovery and audit requirements?

Strong interview framing:

I will design the exchange around a deterministic, single-writer-per-symbol matching engine fed by a sequenced order log. Orders pass pre-trade risk, enter a price-time-priority order book, generate fills, and drive a fair market-data fanout. State is recovered by replaying the sequenced log from snapshots.

## 2. High-Level Design

### 2.1 Architecture

Primary flows:

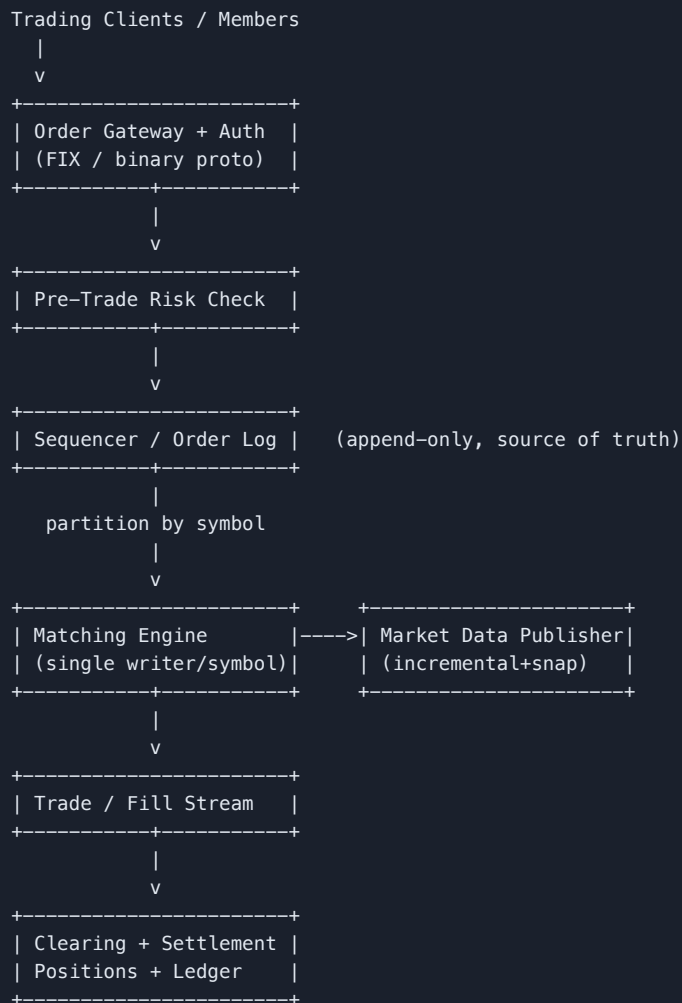
```

Order flow:
Client -> Order Gateway (auth, validate, normalize)
      -> Pre-Trade Risk
      -> Sequencer (assign global order, append to log)
      -> Matching Engine (per symbol)
      -> Trades + book updates
      -> Execution Reports back to client
      -> Market Data fanout to subscribers

Market data flow:
Matching Engine -> Market Data Publisher
                -> incremental feed + periodic snapshot
                -> subscribers (traders, feeds, internal analytics)

Post-trade flow:
Trades -> Clearing -> Settlement -> Positions/Ledger
  
```

Recommended architecture:



Request flow for an order:

1. Client submits order over FIX or a binary protocol to the Order Gateway.
2. Gateway authenticates, validates syntax, and normalizes the order.
3. Pre-Trade Risk verifies limits, buying power, price bands, and fat-finger checks.
4. Sequencer assigns a monotonic sequence number and appends to the durable log.
5. Matching Engine for that symbol consumes the log in order.
6. Engine matches against the resting book by price-time priority.
7. Fills and book deltas are produced deterministically.
8. Execution reports go back to the submitting client.
9. Market Data Publisher fans out incremental updates and periodic snapshots.
10. Trades flow to clearing, settlement, and position/ledger updates.

## 2.2 APIs

### Submit Order

Native protocol is usually FIX or a binary protocol, but a REST-style equivalent:

```
POST /v1/orders
Idempotency-Key: cl-ord-9f2
```

```
{
  "clientOrderId": "cl-ord-9f2",
  "symbol": "AAPL",
  "side": "BUY",
  "type": "LIMIT",
  "price": 191.25,
  "quantity": 100,
  "timeInForce": "GTC"
}
```

Response:

```
{
  "orderId": "ord-100",
  "clientOrderId": "cl-ord-9f2",
  "status": "ACCEPTED",
  "sequence": 84213
}
```

## Cancel Order

```
DELETE /v1/orders/{orderId}
```

## Cancel / Replace (Amend)

```
PUT /v1/orders/{orderId}
```

```
{
  "price": 191.10,
  "quantity": 50
}
```

## Get Order Status

```
GET /v1/orders/{orderId}
Authorization: Bearer <token>
```

## Market Data Subscription

```
SUBSCRIBE symbol=AAPL depth=L2
-> snapshot + incremental deltas over multicast/websocket
```

Important API points:

- Every order carries a `clientOrderId` for idempotency and dedup.
- Order acceptance is not a fill; execution reports carry fill state.
- Cancel is best-effort: the order may have already filled.
- Market data is a stream, not a request-per-quote.

## 2.3 Core Components

---

Think of the exchange as six connected planes:

| Plane             | What it handles              | Main goal                     |
|-------------------|------------------------------|-------------------------------|
| Ingress plane     | gateway, protocol, auth      | validated normalized orders   |
| Risk plane        | pre-trade limits/checks      | prevent unsafe orders         |
| Sequencing plane  | global order + durable log   | deterministic source of truth |
| Matching plane    | order book + matching        | fair, deterministic execution |
| Market data plane | fanout of book/trade updates | fair, fast dissemination      |
| Post-trade plane  | clearing, settlement, ledger | correct positions and money   |

## Component Responsibility Map

| Component             | Owns                       | Does not own       | Scales by              |
|-----------------------|----------------------------|--------------------|------------------------|
| Order Gateway         | protocol, auth, validation | matching logic     | connections/order rate |
| Pre-Trade Risk        | limit/buying-power checks  | order book state   | order rate             |
| Sequencer             | global ordering + log      | matching decisions | log throughput         |
| Matching Engine       | order book, fills          | risk policy        | per-symbol partitions  |
| Market Data Publisher | feed fanout                | matching           | subscriber count       |
| Clearing/Settlement   | netting, positions, money  | matching latency   | trade volume           |
| Recovery/Replay       | snapshots + log replay     | live matching      | symbols/state size     |

## Order Gateway And Protocol

Why it exists:

- Members connect over standardized protocols (FIX, binary).
- Malformed or hostile input must never reach the engine.

Strategy:

- Terminate sessions, authenticate members, enforce message limits.
- Validate and normalize into an internal canonical order.
- Assign/verify `clientOrderId` and reject duplicates.
- Push accepted orders toward risk and sequencing.

Failure behavior:

- Bad message: reject with an execution report, do not forward.
- Session drop: order status is recoverable via query.

Interview signal:

The gateway is a firewall: only well-formed, authenticated, risk-eligible orders reach the engine.

## Pre-Trade Risk

Checks before an order enters the book:

- Buying power / margin / position limits.
- Order size and notional limits (fat-finger guard).
- Price band / collar checks against reference price.
- Self-trade prevention flags.

- Rate limits per member.

Constraints:

- Must be fast (microseconds) and cannot be skipped.
- Deterministic, so replay reproduces the same decisions.

Interview signal:

Risk is on the hot path and must be fast, but skipping it is never acceptable.

## Sequencer And Order Log

Why it exists:

- Determinism requires a single agreed order of events.
- Recovery requires a durable, replayable record.

Strategy:

- Assign a monotonic sequence number to every accepted event.
- Append to a durable, replicated, append-only log.
- The matching engine is a deterministic function of the log.

Failure behavior:

- Engine crash: rebuild by replaying the log from the last snapshot.
- The log, not the in-memory book, is the source of truth.

Interview signal:

The sequenced log is the source of truth; the order book is a materialized view you can always rebuild.

## Matching Engine And Order Book

Order book structure:

- Two sides: bids (descending price) and asks (ascending price).
- Each price level is a FIFO queue of orders (time priority).
- Best bid/ask is the top of book.

Matching rules (continuous trading, price-time priority):

- An incoming buy matches the lowest ask at or below its limit; a sell matches the highest bid at or above its limit.
- Within a price level, oldest resting order fills first.
- Partial fills leave residual quantity resting (unless IOC/FOK).
- Market orders take liquidity until filled or book is exhausted.

Design choices:

| Choice                     | Fit                  | Trade-off                      |
|----------------------------|----------------------|--------------------------------|
| Single writer per symbol   | determinism/fairness | one core per hot symbol        |
| In-memory book             | latency              | needs log-based recovery       |
| Array of price levels      | fast top-of-book     | bounded/known tick range       |
| Map of price -> FIFO queue | flexible             | pointer chasing / cache misses |

Interview signal:

The engine is a single-threaded deterministic loop per symbol. Concurrency lives around it (ingress, fanout), not inside the matching decision.

## Order Types And Time-In-Force

| Type                      | Behavior                                         |
|---------------------------|--------------------------------------------------|
| Limit                     | rest at price if not immediately matchable       |
| Market                    | take liquidity at best available prices          |
| IOC (Immediate-or-Cancel) | fill what it can now, cancel the rest            |
| FOK (Fill-or-Kill)        | fill fully now or cancel entirely                |
| Stop                      | activates to market/limit when trigger price hit |
| Iceberg                   | shows only part of quantity; refills on fill     |

Interview signal:

Time-in-force and order type change matching behavior; state the exact semantics before coding the match loop.

## Market Data Fanout

Feed design:

- Incremental feed: every book delta and trade, sequenced.
- Snapshot feed: periodic full book image for late joiners/recovery.
- Subscribers apply snapshot then incrementals by sequence.

Fairness:

- All subscribers of a feed should receive updates with equal structural opportunity (often multicast).
- No internal subscriber gets an ordering advantage.

Interview signal:

Market data is snapshot + incremental by sequence number; the sequence lets a client detect gaps and re-sync.

## Clearing And Settlement

Post-trade flow:

- Match produces a trade with buyer, seller, price, quantity.
- Clearing nets obligations (often via a central counterparty).
- Settlement moves securities and cash (T+1/T+2 traditionally).
- Positions and ledgers are updated and reconciled.

Interview signal:

Matching is microseconds; settlement is a separate, slower, strongly-consistent pipeline. Keep them decoupled.

## 2.4 Data Layer

---

### Core Data Models

Order:

```
{
  "orderId": "ord-100",
  "clientOrderId": "cl-ord-9f2",
  "symbol": "AAPL",
  "side": "BUY",
  "type": "LIMIT",
  "price": 191.25,
  "quantity": 100,
  "remaining": 100,
  "timeInForce": "GTC",
  "status": "OPEN",
  "sequence": 84213,
  "timestamp": "2026-07-01T13:30:00.000123Z"
}
```

Price level (order book entry):

```
{
  "symbol": "AAPL",
  "side": "BID",
  "price": 191.25,
  "orders": ["ord-100", "ord-105", "ord-111"],
  "totalQuantity": 450
}
```

Trade (fill):

```
{
  "tradeId": "trd-500",
  "symbol": "AAPL",
  "price": 191.25,
  "quantity": 100,
  "buyOrderId": "ord-100",
  "sellOrderId": "ord-090",
  "sequence": 84214,
  "timestamp": "2026-07-01T13:30:00.000200Z"
}
```

Position:

```
{
  "accountId": "acct-7",
  "symbol": "AAPL",
  "quantity": 300,
  "avgPrice": 190.80
}
```

## Storage And Schema Choices

| Data type           | Candidate store           | Why                     |
|---------------------|---------------------------|-------------------------|
| Order log           | append-only sequenced log | source of truth, replay |
| Live order book     | in-memory structures      | ultra-low latency       |
| Order/trade history | relational/columnar store | audit and reporting     |
| Positions/ledger    | strongly consistent DB    | money correctness       |
| Market data feed    | multicast/stream          | fast fanout             |
| Snapshots           | durable object/log store  | recovery                |

Relational-style tables (for history/audit, not the hot path):

```
orders(order_id PK, client_order_id, account_id, symbol, side, type,
       price, quantity, remaining, tif, status, sequence, created_at)
trades(trade_id PK, symbol, price, quantity, buy_order_id, sell_order_id,
       sequence, created_at)
positions(account_id, symbol, quantity, avg_price, PRIMARY KEY(account_id, symbol))
ledger(entry_id PK, account_id, symbol, delta_qty, delta_cash, trade_id, created_at)
```

Important indexes:

- `orders(client_order_id)` for idempotency/dedup.
- `orders(account_id, status)` for open-order queries.
- `trades(symbol, sequence)` for feed replay and reporting.
- `positions(account_id, symbol)` for post-trade updates.

## Partitioning

- Partition matching by symbol; each symbol has a single writer.
- Assign symbols to engine shards/cores by load.
- Hot symbols may get a dedicated core/host.
- The sequenced log is partitioned per symbol (or per shard) to preserve per-symbol order.

## Replication And Consistency

- Order log is replicated for durability before the engine acts (or with tight guarantees).
- The order book is a deterministic materialized view; consistency comes from log order.
- Positions/ledger require strong consistency.
- Market data is eventually consistent but gap-detectable via sequence numbers.

## 2.5 Scalability

### Horizontal Scaling

- Gateways scale by connections and message rate.
- Risk scales with order rate (stateless-ish per account cache).
- Sequencer/log throughput is the global ceiling; shard by symbol.
- Matching scales by adding symbol shards, not by threading a single book.
- Market data scales by fanout tier / multicast trees.
- Clearing/settlement scales by trade volume, asynchronously.

## Hot Symbol Strategy

- Pin hot symbols to dedicated engine cores/hosts.
- Keep the book cache-friendly (array of price levels near the touch).
- Batch market-data updates without violating ordering.
- Apply per-member rate limits and message credits.
- Use circuit breakers / auctions to absorb extreme volatility.

## 2.6 Performance

---

### Latency Budget Example

| Stage                  | Target                          |
|------------------------|---------------------------------|
| Gateway parse/validate | sub-microsecond to microseconds |
| Pre-trade risk         | sub-microsecond to microseconds |
| Sequencing/log append  | microseconds                    |
| Matching decision      | sub-microsecond to microseconds |
| Execution report out   | microseconds                    |
| Market data publish    | microseconds                    |

### Optimization Rules

- Keep the hot path single-threaded and allocation-free.
- Use cache-friendly data structures (arrays over pointer chains near the touch).
- Avoid locks on the matching path; use single-writer per symbol.
- Pre-allocate and reuse objects; avoid garbage collection pauses.
- Use kernel-bypass networking / colocation for the lowest tier.
- Measure tail latency (p99.9), not just averages.

## 2.7 Async Systems

---

Use streams/logs for:

- sequenced order events (the backbone)
- trade/fill events
- market data increments and snapshots
- risk limit updates
- clearing/settlement requests
- position/ledger updates
- audit and surveillance events

Queue notes:

- The sequenced log preserves per-symbol order; do not reorder it.
- Post-trade consumers are idempotent by `tradeId`.
- Market data consumers detect gaps by sequence and request snapshots.
- Recovery replays from snapshot + log; consumers must handle replay.

## 2.8 Security, Privacy, And Compliance

---

Security:

- Strong authentication and session control for members.
- Message-rate and credit limits to prevent abuse/DoS.
- Segregated access to order flow and market data.
- Signed/audited configuration and risk-limit changes.

Privacy/compliance:

- Order flow is sensitive; prevent information leakage/front-running.
- Full immutable audit trail for regulators.
- Trade surveillance for spoofing, layering, wash trades.
- Time synchronization (accurate clocks) for sequencing and reporting.

Abuse controls:

- Fat-finger and price-band checks.
- Self-trade prevention.
- Kill switch per member and per symbol.
- Circuit breakers and trading halts.

## 2.9 Observability And Operations

---

Core SLIs:

| Area        | Metrics                                                    |
|-------------|------------------------------------------------------------|
| Gateway     | message rate, reject rate, session drops                   |
| Risk        | check latency, rejections, limit breaches                  |
| Sequencer   | log append latency, throughput, replication lag            |
| Matching    | order-to-ack, order-to-match, book depth, queue length     |
| Market data | publish latency, gap/resync rate, subscriber lag           |
| Post-trade  | trade throughput, settlement failures, position mismatches |

Alerts:

- Order-to-match tail latency exceeds threshold.
- Sequencer/log replication lag rising.
- Market data gaps / resync storms.
- Risk rejection spikes or a member hitting limits repeatedly.
- Book state divergence between primary and replica.
- Settlement/position reconciliation mismatches.

## 2.10 Tradeoffs

| Decision             | Option A                   | Option B              | Trade-off                      |
|----------------------|----------------------------|-----------------------|--------------------------------|
| Matching concurrency | single writer per symbol   | multi-threaded book   | determinism vs raw parallelism |
| Book structure       | array of price levels      | map of price -> queue | latency vs flexible tick range |
| Source of truth      | sequenced log              | in-memory book        | recoverability vs simplicity   |
| Market data          | multicast fair feed        | per-client query      | fairness/scale vs simplicity   |
| Risk                 | inline pre-trade           | async post-trade      | safety vs latency              |
| Durability           | replicate log before match | match then persist    | safety vs latency              |

Interview framing:

I would make the matching engine a deterministic single-writer-per-symbol loop driven by a sequenced, durable log. Ingress, risk, and market-data fanout are concurrent around it. Determinism gives fairness, reproducibility, and clean recovery; scaling is by symbol sharding, not by threading one book.

## 3. Low-Level Design

LLD goal:

Model the exchange around orders, a price-time-priority order book, a deterministic matching engine, trades, positions, and a sequenced event log.

Simple rules:

- One writer per symbol.
- Match by price first, then time.
- The log is truth; the book is a view.
- Every client message is idempotent by `clientOrderId`.

### 3.1 Object Modelling

Core entities:

| Entity                      | Owns                               | Key invariant                         |
|-----------------------------|------------------------------------|---------------------------------------|
| <code>Order</code>          | side, price, qty, remaining, state | remaining never exceeds quantity      |
| <code>PriceLevel</code>     | FIFO queue at a price              | time priority within level            |
| <code>OrderBook</code>      | bid/ask ladders                    | best bid < best ask (no crossed book) |
| <code>MatchingEngine</code> | match loop per symbol              | deterministic price-time matching     |
| <code>Trade</code>          | buy/sell order ids, price, qty     | generated once per match              |
| <code>Position</code>       | account holdings                   | updated once per trade leg            |

Core services:

| Service             | Responsibility       | Should not do             |
|---------------------|----------------------|---------------------------|
| GatewayService      | protocol/validation  | matching                  |
| RiskService         | pre-trade checks     | book mutation             |
| SequencerService    | order + durable log  | matching decisions        |
| MatchingEngine      | match/cancel/replace | risk policy               |
| MarketDataPublisher | feed fanout          | matching                  |
| ClearingService     | netting/settlement   | latency-critical matching |

## 3.2 OOP Fundamentals

Encapsulation:

- OrderBook owns insertion, removal, and best-price math.
- Order owns its remaining-quantity and state transitions.
- PriceLevel owns FIFO ordering within a price.

Abstraction:

- OrderRepository / EventLog hides persistence and replay.
- MarketDataFeed hides multicast/websocket transport.
- RiskPolicy hides specific limit checks.

Polymorphism:

- Different order types ( Limit , Market , IOC , FOK ) implement a common match contract.
- Different matching policies for continuous vs auction.

Composition:

- MatchingEngine composes OrderBook , TradePublisher , and EventLog .

## 3.3 SOLID Principles

| Principle             | Trading system application                                        |
|-----------------------|-------------------------------------------------------------------|
| Single Responsibility | MatchingEngine only matches; risk lives in RiskService            |
| Open/Closed           | add a new order type without rewriting the match loop             |
| Liskov Substitution   | any order type honors the match/cancel contract                   |
| Interface Segregation | separate gateway, risk, matching, market-data APIs                |
| Dependency Inversion  | engine depends on EventLog / Feed interfaces, not implementations |

## 3.4 Design Patterns

| Pattern            | Where to use                  | Why                           |
|--------------------|-------------------------------|-------------------------------|
| Event Sourcing     | sequenced order log           | deterministic replay/recovery |
| State              | order lifecycle               | valid transitions             |
| Strategy           | order type / matching policy  | swap behavior cleanly         |
| Observer/Publisher | market data + trades          | fanout to subscribers         |
| Command            | order/cancel/replace messages | idempotency and replay        |

## 3.5 UML / Diagrams

### Order Match Sequence

```
Client -> GatewayService: submit(order)
GatewayService -> RiskService: check(order)
RiskService -> SequencerService: accept -> assign sequence, append log
SequencerService -> MatchingEngine: onOrder(order)
MatchingEngine -> OrderBook: match against opposite side
MatchingEngine -> TradePublisher: publish trades
MatchingEngine -> Client: execution report
MatchingEngine -> MarketDataPublisher: publish book delta
```

### Cancel Sequence

```
Client -> GatewayService: cancel(orderId)
GatewayService -> SequencerService: append cancel event
SequencerService -> MatchingEngine: onCancel(orderId)
MatchingEngine -> OrderBook: remove resting order (if present)
MatchingEngine -> Client: cancel ack or "too late" (already filled)
MatchingEngine -> MarketDataPublisher: publish book delta
```

## 3.6 Class Design

Interfaces:

```

interface OrderBook {
    List<Trade> add(Order order);           // returns fills generated
    boolean cancel(String orderId);       // false if not resting
    Optional<Order> replace(String orderId, long newPrice, long newQty);
    long bestBid();
    long bestAsk();
}

interface RiskPolicy {
    RiskResult check(Order order, AccountState account);
}

interface EventLog {
    long append(Event event);             // returns sequence number
    void replay(EventConsumer consumer);
}

interface MarketDataPublisher {
    void publishDelta(BookDelta delta);
    void publishTrade(Trade trade);
    void publishSnapshot(BookSnapshot snapshot);
}

```

Design notes:

- `add()` performs price-time matching and returns fills; residual rests.
- `cancel()` is best-effort: an order may already be fully filled.
- `replace()` typically loses time priority when price/qty increases.
- `append()` must be durable before the engine acts on the event.

### 3.7 Edge Cases

| Case                         | Handling                                                            |
|------------------------------|---------------------------------------------------------------------|
| cancel arrives after fill    | return "too late"; order already gone                               |
| crossed/locked book          | matching engine must resolve immediately, never rest a crossed book |
| self-trade                   | self-trade prevention cancels/decrements per policy                 |
| market order into empty book | reject or cancel remainder per rules                                |
| FOK cannot fully fill        | cancel entirely, no partial                                         |
| duplicate clientId           | idempotent: return existing order state                             |
| replace during partial fill  | apply to remaining qty; adjust priority                             |
| engine crash mid-batch       | replay log from snapshot to exact state                             |
| stale market data (gap)      | subscriber re-syncs from snapshot                                   |

## 4. Machine Coding Layer

### 4.1 Code Structure

Suggested package layout:

```
exchange/  
  domain/  
    Order.java  
    PriceLevel.java  
    OrderBook.java  
    Trade.java  
    Position.java  
  service/  
    GatewayService.java  
    RiskService.java  
    SequencerService.java  
    MatchingEngine.java  
    MarketDataPublisher.java  
  port/  
    EventLog.java  
    OrderRepository.java  
    MarketDataFeed.java  
  adapter/  
    InMemoryEventLog.java  
  app/  
    ExchangeDemo.java
```

## 4.2 Core Logic Implementation

---

Focused Python implementation of a price-time-priority limit order book:

```

from dataclasses import dataclass, field
from enum import Enum
from itertools import count
from typing import Dict, List, Optional
import heapq

class Side(str, Enum):
    BUY = "BUY"
    SELL = "SELL"

@dataclass
class Order:
    order_id: int
    side: Side
    price: int          # integer ticks avoid float errors
    quantity: int
    remaining: int
    seq: int            # arrival sequence for time priority

@dataclass
class Trade:
    buy_order_id: int
    sell_order_id: int
    price: int
    quantity: int

class OrderBook:
    """Price-time priority limit order book for one symbol (single writer)."""

    def __init__(self) -> None:
        # bids: max-heap by price, then min seq (time). Use negatives for max.
        self._bids: List = [] # (-price, seq, order_id)
        self._asks: List = [] # (price, seq, order_id)
        self._orders: Dict[int, Order] = {}
        self._seq = count(1)
        self._id = count(1)

    def _next_seq(self) -> int:
        return next(self._seq)

    def submit_limit(self, side: Side, price: int, quantity: int) -> List[Trade]:
        order = Order(next(self._id), side, price, quantity, quantity, self._next_seq())
        trades = self._match(order)
        if order.remaining > 0:
            self._rest(order)
        return trades

    def _match(self, order: Order) -> List[Trade]:
        trades: List[Trade] = []
        if order.side == Side.BUY:
            book = self._asks
            crosses = lambda best_price: best_price <= order.price
        else:
            book = self._bids
            crosses = lambda best_price: -best_price >= order.price

        while order.remaining > 0 and book:
            best = book[0]
            best_price = best[0] if order.side == Side.BUY else -best[0]
            price = best_price if order.side == Side.BUY else -best_price
            if not crosses(best_price):
                break
            resting = self._orders.get(best[2])
            if resting is None or resting.remaining == 0:
                heapq.heappop(book) # stale entry, drop it

```

```

        continue
    fill = min(order.remaining, resting.remaining)
    order.remaining -= fill
    resting.remaining -= fill
    if order.side == Side.BUY:
        trades.append(Trade(order.order_id, resting.order_id, price, fill))
    else:
        trades.append(Trade(resting.order_id, order.order_id, price, fill))
    if resting.remaining == 0:
        heapq.heappop(book)
        self._orders.pop(resting.order_id, None)
    return trades

def _rest(self, order: Order) -> None:
    self._orders[order.order_id] = order
    if order.side == Side.BUY:
        heapq.heappush(self._bids, (-order.price, order.seq, order.order_id))
    else:
        heapq.heappush(self._asks, (order.price, order.seq, order.order_id))

def cancel(self, order_id: int) -> bool:
    order = self._orders.pop(order_id, None)
    if order is None:
        return False        # already filled or unknown
    order.remaining = 0      # lazy removal; heap entry dropped when popped
    return True

def best_bid(self) -> Optional[int]:
    while self._bids and self._orders.get(self._bids[0][2]) is None:
        heapq.heappop(self._bids)
    return -self._bids[0][0] if self._bids else None

def best_ask(self) -> Optional[int]:
    while self._asks and self._orders.get(self._asks[0][2]) is None:
        heapq.heappop(self._asks)
    return self._asks[0][0] if self._asks else None

book = OrderBook()
book.submit_limit(Side.SELL, price=100, quantity=10)    # ask 100 x10
book.submit_limit(Side.SELL, price=101, quantity=5)    # ask 101 x5
fills = book.submit_limit(Side.BUY, price=100, quantity=7) # buys 7 @ 100
print(fills)                                           # one trade: 7 @ 100
print(book.best_ask())                                # 100 (3 left) still resting

```

## 4.3 Data Structures

| Structure                      | Use                    |
|--------------------------------|------------------------|
| heap/tree of price levels      | best bid/ask retrieval |
| FIFO queue per price level     | time priority          |
| dict[orderId -> Order]         | O(1) cancel/lookup     |
| append-only log                | sequencing and replay  |
| dict[clientOrderId -> orderId] | idempotency/dedup      |

## 4.4 Concurrency

High-signal concurrency issues:

- Many orders arrive for the same symbol simultaneously.
- Cancel racing with a fill on the same order.

- Market data fanout must not block the match loop.
- Recovery replay must not interleave with live matching.

Handling strategy:

- Single writer per symbol; ingress hands off via a lock-free queue.
- Lazy cancel with tombstones to avoid heap-delete cost.
- Publish market data from a separate consumer of the trade/delta stream.
- Freeze the symbol during replay, then resume live from the log tail.

## 4.5 Testing Thinking

---

Unit tests:

- Buy crossing the spread fills at resting ask price.
- Time priority: earlier order at same price fills first.
- Partial fill leaves correct residual resting.
- FOK cancels fully when it cannot fully fill.
- Cancel after fill returns "too late".
- Replay of the log reproduces the exact book.

Load tests:

- Quote storm (adds/cancels) on one hot symbol.
- Market-open auction burst.
- News-shock volatility with circuit breaker.
- Market-data subscriber gap/resync under load.

## 5. Handling Abnormal Traffic Spikes

---

### 5.1 Spike Types

---

| Spike type            | Example                  | Main risk                 |
|-----------------------|--------------------------|---------------------------|
| Market open/close     | opening/closing auction  | matching + data burst     |
| News shock            | earnings, macro event    | quote storm, volatility   |
| Quote stuffing        | abusive add/cancel flood | engine + feed overload    |
| Fat-finger order      | huge erroneous order     | price dislocation         |
| Feed subscriber storm | mass resync after gap    | market-data amplification |

### 5.2 Immediate Spike Response

---

1. Enforce per-member message credits/rate limits.
2. Apply price bands and reject out-of-band orders.
3. Trigger circuit breakers / trading halts on extreme moves.
4. Shed non-critical traffic (analytics feeds) before core order flow.
5. Keep the sequenced log durable; never drop accepted events.
6. Batch market-data publishing without reordering.

7. Isolate hot symbols on dedicated engine cores.

## 5.3 Degradation Policy

---

Protect in this order:

1. Determinism and fairness (never reorder).
2. Durability of accepted orders/trades.
3. Order acceptance and matching for core symbols.
4. Execution reports to members.
5. Market data freshness.
6. Analytics/non-critical feeds.

Not allowed:

- Violate price-time priority.
- Lose or duplicate an accepted order or trade.
- Cross the book or match at the wrong price.
- Skip pre-trade risk to save latency.

## 5.4 Spike Interview Answer

---

During spikes I protect determinism and durability first. Members get message credits and price bands; extreme moves trigger circuit breakers. The sequenced log never drops accepted events, market data is batched without reordering, and hot symbols are isolated on dedicated cores. Nothing is allowed to break price-time fairness.

# 6. Scaling To A Global Venue

---

## 6.1 Global Architecture

---

```
Global venue
-> regional gateways + colocation
-> per-symbol sequenced logs
-> matching engines sharded by symbol
-> market data fanout (multicast trees / regional relays)
-> post-trade clearing and settlement pipeline
```

## 6.2 Multi-Region Strategy

---

- Keep each symbol's matching in a single owner/region for determinism.
- Colocate members near the matching engine for fairness at the top tier.
- Replicate the order log for durability and hot standby.
- Fan out market data via regional relays with sequence numbers.
- Run clearing/settlement as a separate strongly-consistent pipeline.

## 6.3 Global Capacity Plan

| Layer         | Scaling plan                         |
|---------------|--------------------------------------|
| Gateway       | regional pods, connection sharding   |
| Risk          | per-account cached limits, sharded   |
| Sequencer/log | per-symbol partitions, replicated    |
| Matching      | symbol sharding, dedicated hot cores |
| Market data   | multicast/relay fanout tiers         |
| Clearing      | async netting by trade volume        |
| Recovery      | snapshots + log replay per shard     |

## 6.4 Global Interview Answer

I scale a global exchange by sharding matching per symbol, each a deterministic single-writer engine over a replicated sequenced log. Gateways and colocation handle fairness and latency at the edge; market data fans out via sequenced multicast relays; clearing and settlement run as a decoupled strongly-consistent pipeline.

## Gold-Level Interview Traps

Watch for these mistakes when presenting this design:

- Designing only the happy path and ignoring retries, timeouts, and partial failure.
- Skipping the data model or not naming the source of truth (it is the sequenced log, not the book).
- Multi-threading a single order book and breaking determinism/fairness.
- Treating market data as request-per-quote instead of snapshot + incremental.
- Scaling every component equally instead of sharding by symbol.
- Forgetting idempotency, sequencing, and recovery/replay.
- Giving a complex final design without first stating the simple single-symbol MVP.

## 7. Final Interview Playbook

Use this answer flow:

```
I will clarify exchange vs participant, order types, continuous vs auction, market data depth, and latency/fairness
I will estimate order/cancel rates, hot-symbol concentration, trades/day, and market-data fanout.
HLD includes Gateway, Risk, Sequencer/Log, Matching Engine, Market Data Publisher, and Clearing/Settlement.
The matching engine is a deterministic single-writer-per-symbol loop over a sequenced durable log.
Matching is price-time priority; the log is the source of truth and enables replay recovery.
Market data is snapshot + incremental by sequence.
Scaling is by symbol sharding; spikes are handled with credits, price bands, and circuit breakers.
```

## 8. Fast Recall Rules

- The sequenced log is truth; the order book is a materialized view.

- One writer per symbol; concurrency lives around the engine, not inside it.
- Match by price first, then time (price-time priority).
- Never rest a crossed book.
- Pre-trade risk is on the hot path and cannot be skipped.
- Market data is snapshot + incremental by sequence number.
- Cancel is best-effort; the order may already be filled.
- Use integer ticks, not floats, for prices.
- Measure tail latency (p99.9), not averages.
- Matching is microseconds; settlement is a separate slower pipeline.